

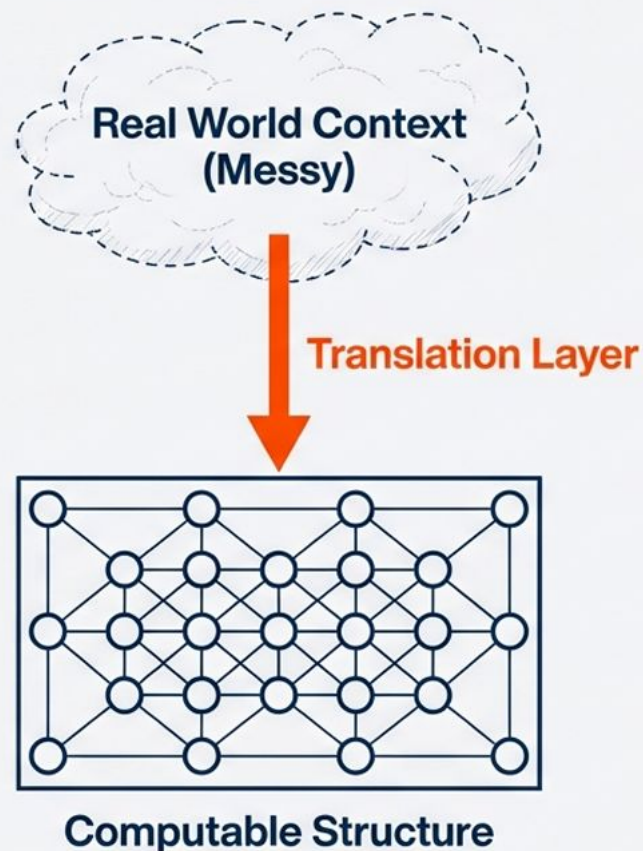
The Imperative of Problem Formulation

Translating reality into syntax.

Problem formulation is the process of clearly defining a problem so that an artificial intelligence agent can solve it. It acts as the translation layer between a 'Real-World Problem' and an 'AI Problem'.

The Why: If a problem is not defined properly, the AI cannot solve it correctly. Poor formulation leads to inefficient search and the risk that a solution may never be found.

**Good Formulation =
Faster + Better Solutions**

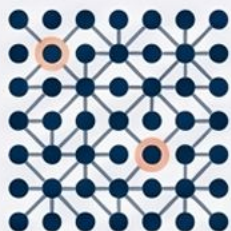


The Five Components of AI Problem Formulation



1. Initial State

The starting position of the agent.



2. State Space

The universe of all possible states reachable from the initial state.



3. Actions (Operators)

The specific moves allowed to transition between states.



4. Goal State

The desired result or condition that signifies success.

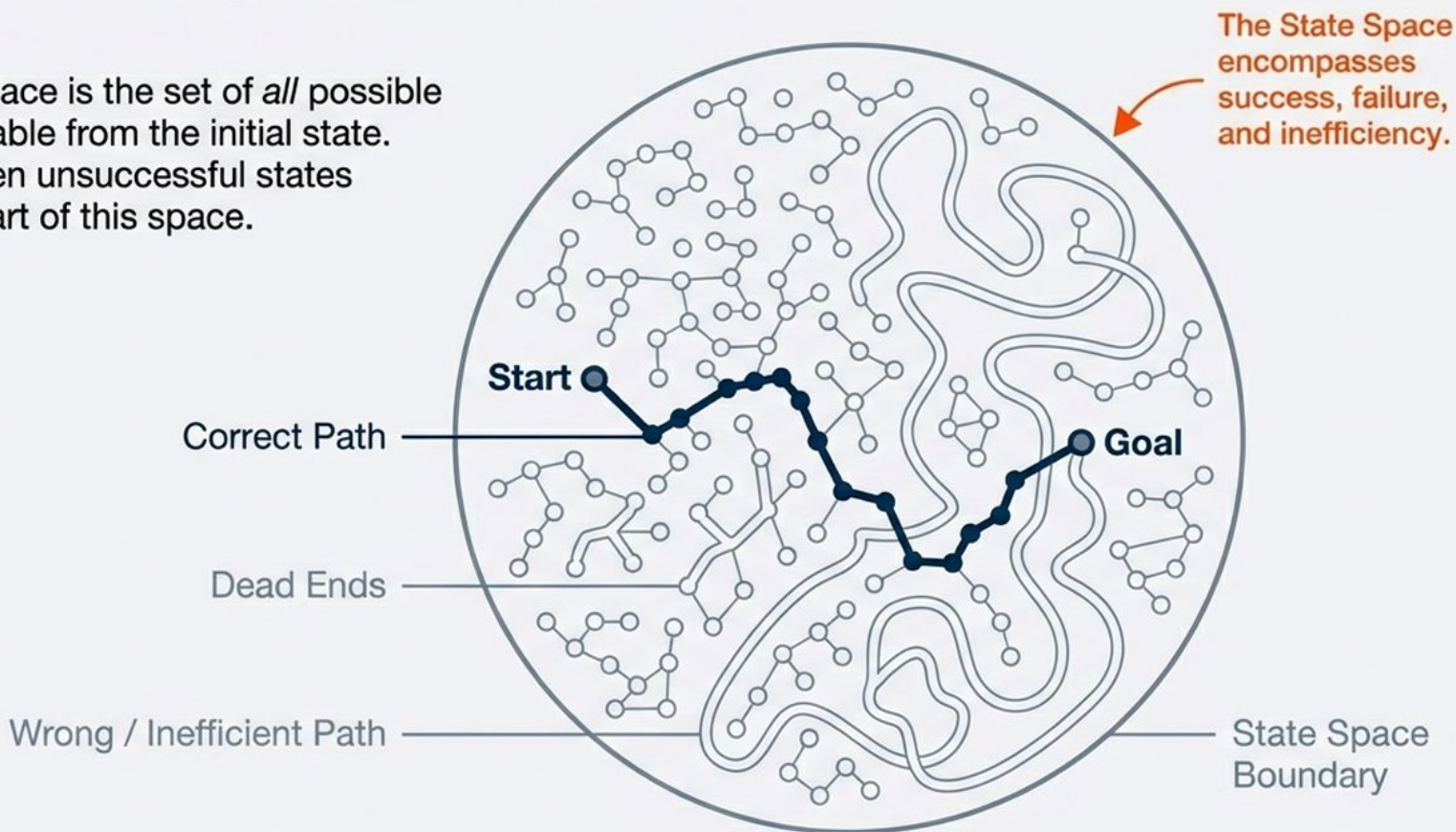


5. Path Cost

A metric to measure the 'cost' of the path taken to reach the goal.

Blueprinting the State Space

The State Space is the set of *all* possible states reachable from the initial state. Crucially, even unsuccessful states are a valid part of this space.



Theory in Action: The Water Jug Problem

The Physical Reality



The Challenge: Measure exactly 2 Litres using only these two jugs.

The AI Formulation

Initial State: $(0,0)$ — Both jugs are empty.

State Space: All possible water combinations (x, y) .

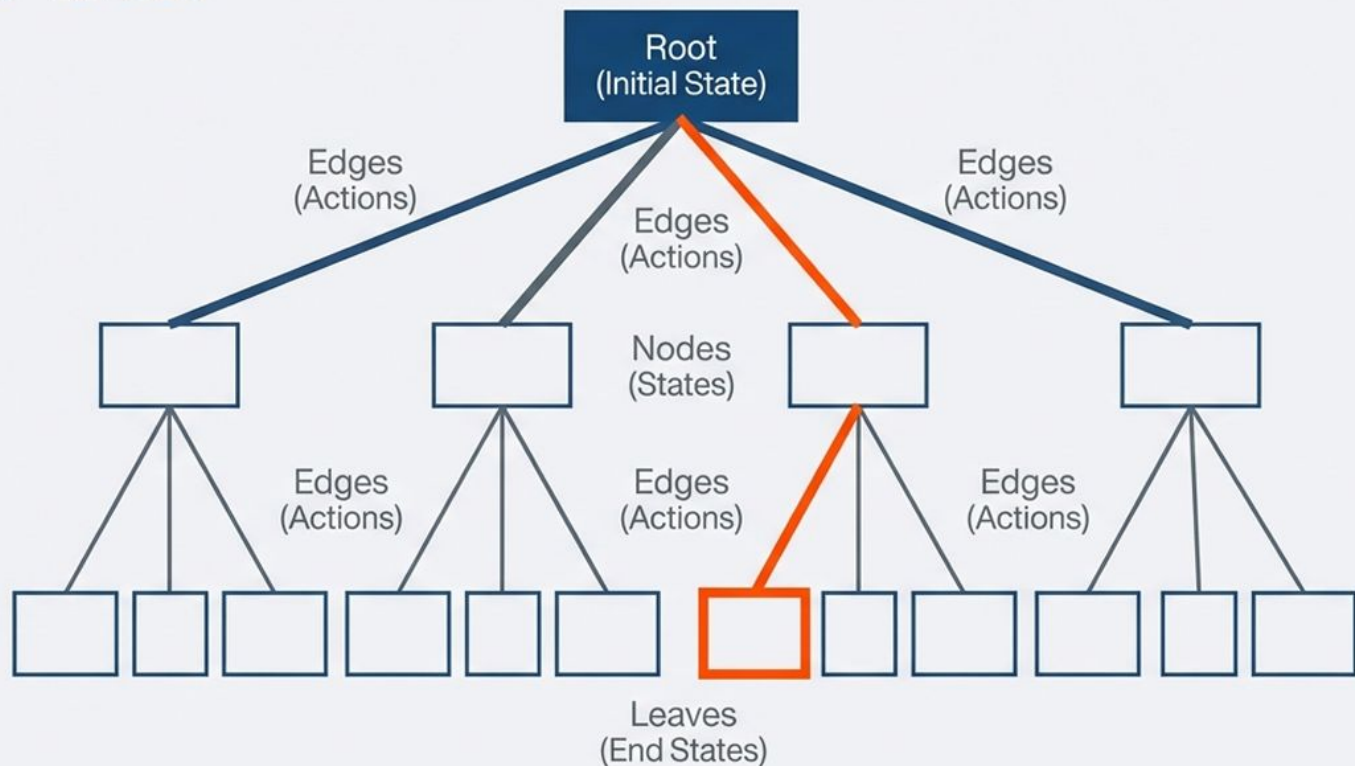
Example State: $(3,1)$ — 3L in the big jug, 1L in the small jug.

Actions: Fill, Empty, Pour.

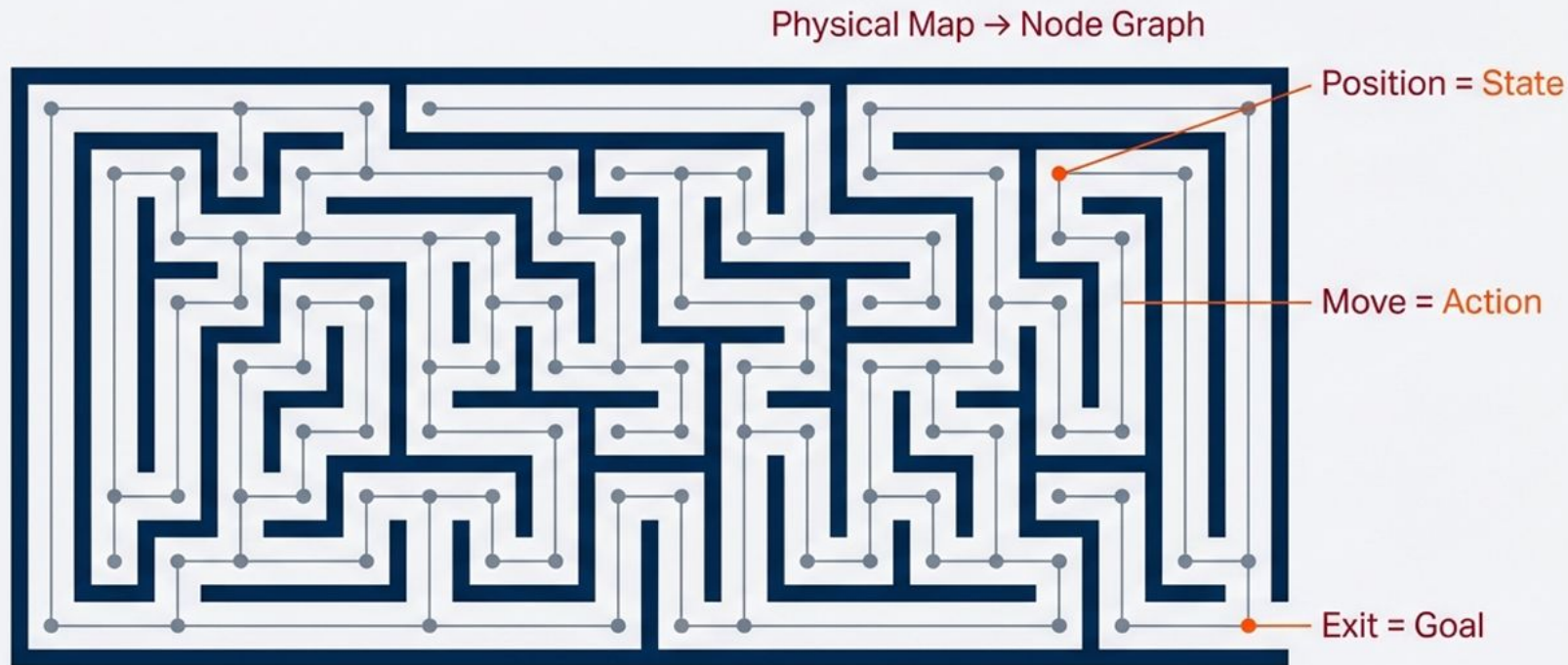
Goal State: Any state (x,y) where $x=2$ or $y=2$.

Visualizing the Process: The Search Tree

State Space Search is the process of exploring the space to find a path from the Initial State to the Goal State. The AI starts at the root, tries different actions, and stops only when the goal is found.

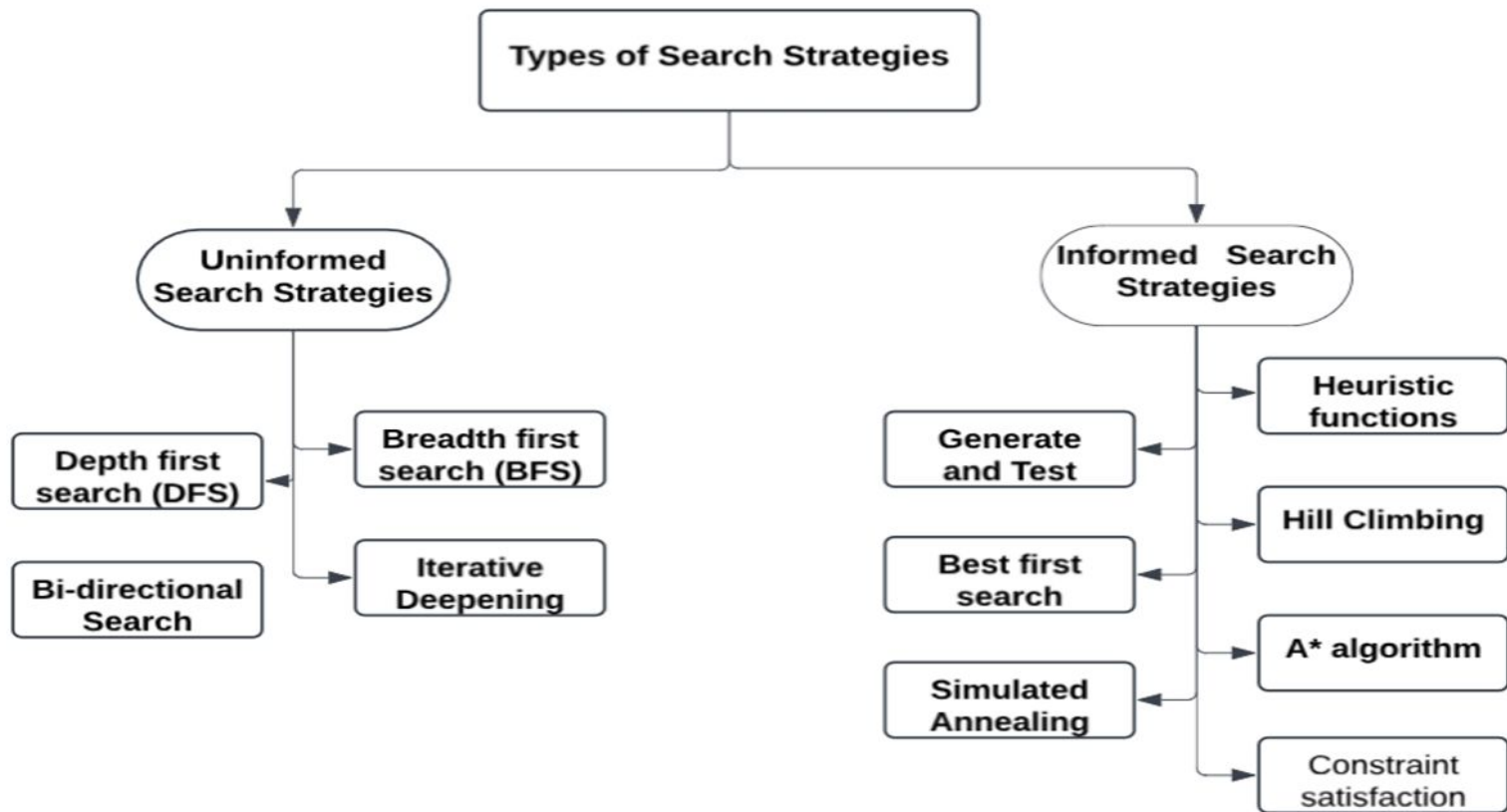


Theory in Action: The Maze



The AI explores paths until it **locates the exit**. Crucially, **some paths fail**—but they must still be explored by the algorithm to confirm they are failures.

Types of Search Strategies:



Introduction to Uninformed Search

Navigating without a map.



Definition: Uninformed Search (or 'Blind Search')

is an algorithmic approach where the agent has no extra information about the goal's location.

Inputs Only:

1. Initial State
2. Actions
3. Goal Test

The Analogy:

It is comparable to searching for a lost key in a dark room—systematically checking every area without knowing where the key is likely to be.

Operational Characteristics

Why systems are termed 'Blind'.

System Behavior:

- No heuristics.
- No guidance.
- Systematic exploration.

Simplicity

Simple to implement.
Requires minimal setup.

Knowledge

No Domain Knowledge.
Operates purely on graph structure.

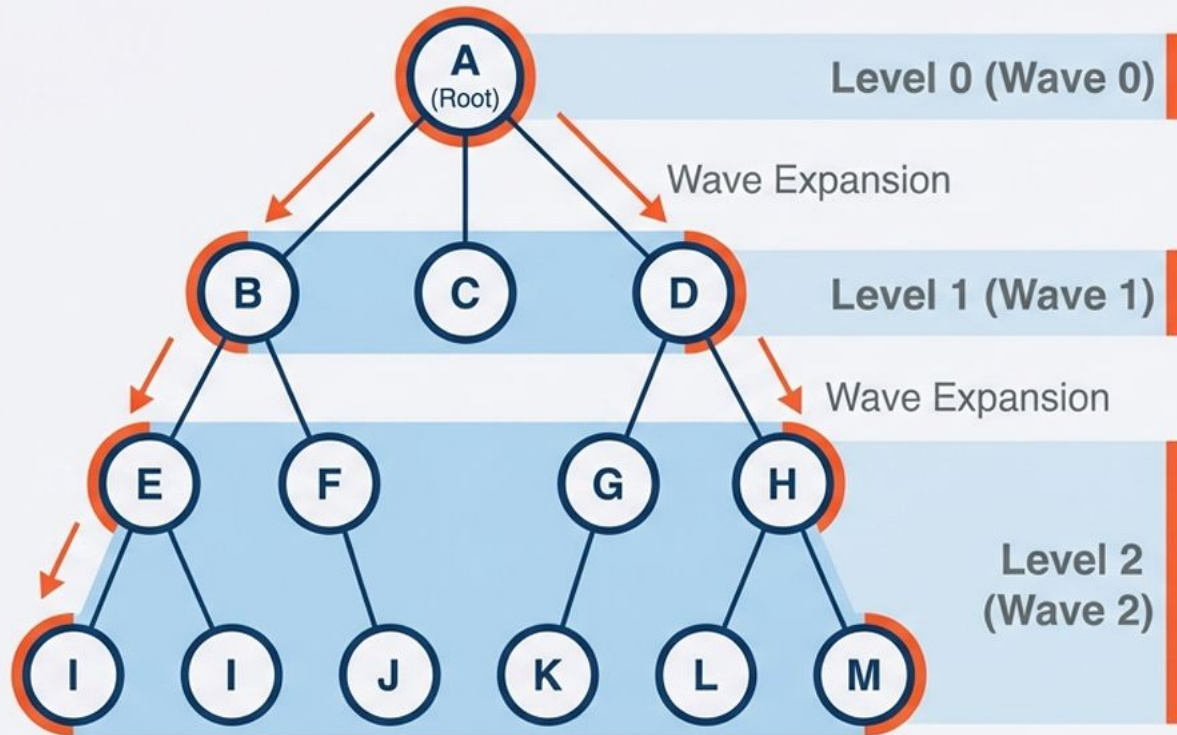
Effort

High Effort.
May explore many unnecessary states.

Reliability

Completeness.
Guaranteed to find a solution in some algorithms.

Deep Dive: Breadth First Search (BFS)



The Mechanics

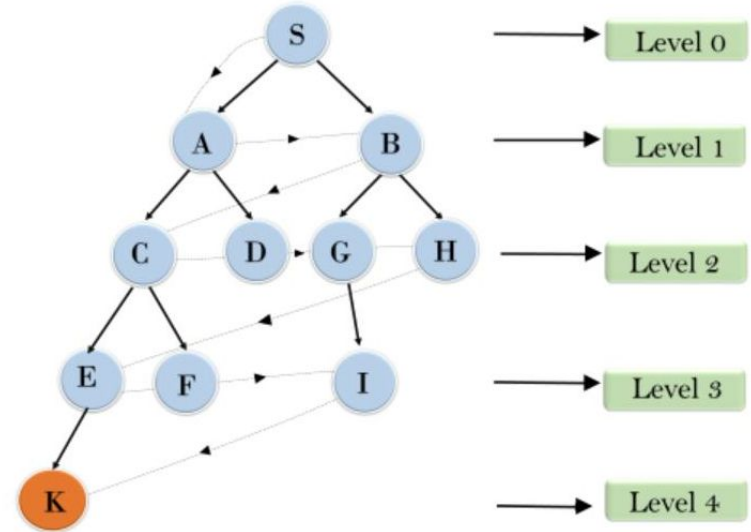
- Explores the tree level by level.
- Utilizes a Queue data structure (First In, First Out).

Strengths

- Guaranteed to find the Shortest Path.
- Ideal for small, shallow problems.

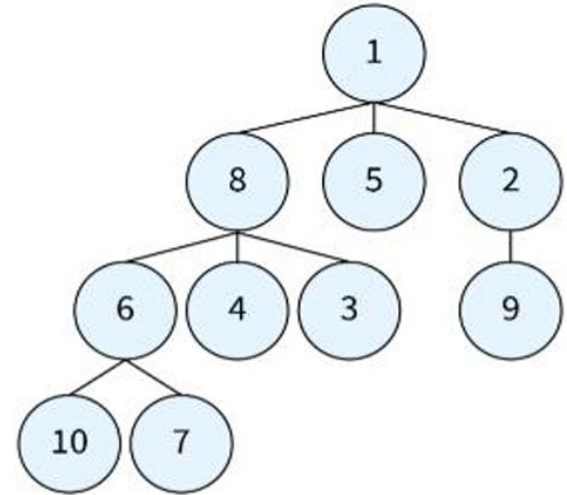
Breadth-First Search (BFS)

- Explores level by level.
- Expands nodes by increasing depth
- Find shortest path in unweighted graph
- Implemented using FIFO queue data structure.
- Time complexity: $O(V+E)$
- Space complexity: $O(V)$



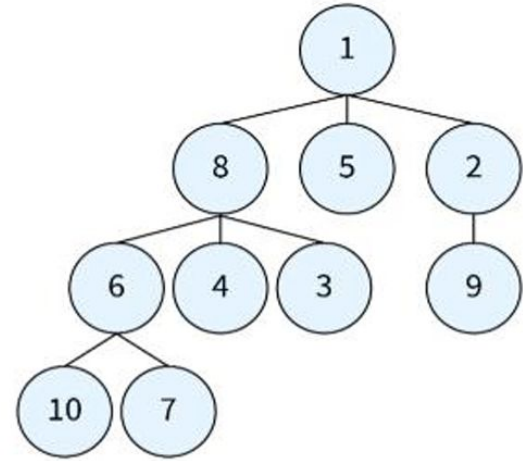
Algorithm of Breadth-First Search (BFS)

- Create an empty queue, enqueue root.
- While queue isn't empty, continue process.
- Dequeue the front node from queue.
- Check if dequeued node is goal.
- If not, enqueue unvisited neighbors.
- Repeat until goal found or queue empty.



Algorithm of Breadth-First Search (BFS)

- Create an empty queue, enqueue root.
- While queue isn't empty, continue process.
- Dequeue the front node from queue.
- Check if dequeued node is goal.
- If not, enqueue unvisited neighbors.
- Repeat until goal found or queue empty.



BFS : 1 8 5 2 6 4 3 9 10 7

Pseudocode for BFS:

Input: s as the source node

```
BFS (G, s)
let Q be queue.
Q.enqueue( s )
```

```
mark s as visited
while ( Q is not empty)
v = Q.dequeue( )
```

```
for all neighbors w of v in Graph G
if w is not visited
Q.enqueue( w )
mark w as visited
```

Advantages of BFS:

- Finds the shortest path fast
- Explores all possibilities
- Simple and easy to implement
- Guarantees complete solution
- Good for shallow searches
- Suitable for unweighted graphs

Disadvantages of BFS:

- High memory consumption
- Slow for large graphs
- Inefficient for weighted graphs
- Requires additional storage space
- Fails with infinite paths
- Expensive computationally at scale

Task: Find path from City A City G.

